Práctica 1

Guion 1

Búsqueda Local y Trayectorias: El Viajante de Comercio (TSP)

Metaheurísticas (Curso 2026/2027)

Algoritmos implementados:

Greedy · Greedy aleatorio · Búsqueda local del primer mejor (DLB) · Búsqueda tabú

Autores:

Alejandro Ríos Cabello	arc00166	DNI: XXXXXXXXX
Nicolás Rafael Chelaru Tanase	nrct0001	DNI: XXXXXXXXX

Grupo: XX

Universidad de Jaén

18 de octubre de 2026

Índice

1 Descripción del problema **3**
1.1 El problema del viajante de comercio . . . 3
1.2 Representación de las soluciones . . . 3
1.3 Función objetivo . . . 3
1.4 Operadores comunes . . . 3
1.5 Descripción del código . . . 4

2 Algoritmos **4**
2.1 Greedy . . . 4
2.2 Greedy aleatorio . . . 5
2.3 Búsqueda local del primer mejor con DLB . . . 5
2.4 Búsqueda tabú . . . 6

3 Experimentos y análisis de resultados **9**
3.1 Parámetros y semillas . . . 9
3.2 Resultados . . . 9
3.3 Análisis de los algoritmos . . . 11
3.4 Tiempos, desviaciones e influencia del tamaño . . . 15
3.5 Análisis de la mejor solución . . . 16
3.6 Ficheros de registro . . . 17

4 Referencias **17**

Anexo **18**

A Carpeta de Logs **18**

Índice de Figuras

1 Desviación media respecto al óptimo por algoritmo e instancia (escala logarítmica). 11
2 Distribución de la desviación de las ejecuciones del greedy aleatorio (GA), la búsqueda local (BL) y la búsqueda tabú. Cada punto es una ejecución. . 11
3 Convergencia de la mejor ejecución de la búsqueda tabú y de la búsqueda local en ch130, a280 y u1060 (coste en escala logarítmica). La línea fina es la solución actual del tabú y la discontinua, el óptimo. 14
4 Tiempo medio por ejecución frente al número de ciudades (escalas logarítmicas). 15
5 Mejor recorrido de la búsqueda local y de la búsqueda tabú en ch130. . . . 16

Índice de Tablas

1 Parámetros de las ejecuciones. 9
2 Resultados de Greedy 10
3 Resultados de Greedy aleatorio 10
4 Resultados de Búsqueda local 10
5 Resultados de Búsqueda tabú 10
6 Desviación media respecto al óptimo y tiempo medio por algoritmo e instancia. 11
7 Movimientos realizados por la búsqueda local y motivo de parada. 12
8 Comportamiento interno de la búsqueda tabú (10.000 iteraciones por ejecución). 13

1. Descripción del problema

1.1. El problema del viajante de comercio

El problema del viajante de comercio (TSP, *Travelling Salesman Problem*) consiste en, dado un conjunto de n ciudades y la distancia entre cada par de ellas, encontrar el recorrido cerrado de longitud mínima que visita cada ciudad exactamente una vez y vuelve a la ciudad de origen. Es un problema NP-duro: el número de recorridos distintos crece como $(n-1)!/2$, por lo que para las instancias usadas (de 130 a 18.512 ciudades) es inviable una búsqueda exhaustiva y se recurre a metaheurísticas.

Las instancias proceden de TSPLIB (`ch130`, `pr144`, `a280`, `u1060` y `d18512`). Cada fichero contiene las coordenadas (x_i, y_i) de las ciudades y el problema es **simétrico**: la distancia entre dos ciudades es la euclídea,

$$d(i,j) = d(j,i) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}. \tag{1}$$

Al leer el fichero se calcula una sola vez la matriz de distancias D de tamaño $n \times n$, de forma que consultar una distancia durante la búsqueda cuesta $O(1)$.

1.2. Representación de las soluciones

Una solución es una **permutación** $\pi = (\pi_0, \pi_1, \ldots, \pi_{n-1})$ de las ciudades $\{0, \ldots, n-1\}$: π_k es la ciudad visitada en la posición k del recorrido. Tras la última ciudad se vuelve a la primera, así que el recorrido es circular. Esta representación garantiza por construcción que cada ciudad aparece una sola vez, y todos los operadores empleados transforman una permutación en otra permutación.

1.3. Función objetivo

Se minimiza la longitud total del recorrido:

$$F(\pi) = \sum_{k=0}^{n-1} d\left(\pi_k, \pi_{(k+1) \bmod n}\right). \tag{2}$$

Evaluar una solución completa cuesta $O(n)$. Como medida de calidad se usa la desviación respecto al óptimo conocido de cada instancia:

$$\mathrm{Desv}(\pi) = 100 \cdot \frac{F(\pi) - F^*}{F^*}. \tag{3}$$

1.4. Operadores comunes

Operador de intercambio (2-opt del guion). Dadas dos posiciones $i \neq j$, se intercambian las ciudades que ocupan: $\pi_i \leftrightarrow \pi_j$. El entorno de una solución contiene $n(n-1)/2$ vecinos.

Evaluación factorizada. Al intercambiar dos posiciones solo cambian los arcos que tocan a esas posiciones, por lo que el coste del vecino se obtiene en $O(1)$ a partir del coste actual:

$$F(\pi') = F(\pi) - \text{arcos que desaparecen} + \text{arcos nuevos}. \tag{4}$$

Si i y j no son adyacentes desaparecen los cuatro arcos (π_{i-1}, π_i), (π_i, π_{i+1}), (π_{j-1}, π_j), (π_j, π_{j+1}) y aparecen los cuatro que unen esos vecinos con la ciudad intercambiada. Si son adyacentes (por ejemplo $j = i + 1$), el arco (π_i, π_j) no desaparece sino que se invierte, y solo cambian los dos arcos exteriores. Los índices se toman módulo n, de manera que el caso de la primera y la última posición (que también son adyacentes en un recorrido circular) se trata igual que cualquier otro par de posiciones contiguas.

Don't Look Bits (DLB). Es un vector de n bits, uno por posición. Un bit a 0 indica que merece la pena explorar los intercambios de esa posición; un bit a 1, que ya se exploraron todos sin encontrar mejora. Al empezar todos están a 0; cuando una posición agota sus movimientos sin mejora se pone a 1, y cuando participa en un movimiento que se aplica se vuelve a poner a 0. Así la búsqueda se concentra en las zonas del recorrido que han cambiado recientemente.

Generación de semillas. Partiendo de la semilla `1658743`, cada ejecución usa la anterior rotada un dígito a la izquierda ($1658743 \to 6587431 \to 5874316 \to \dots$). La semilla inicializa el generador `random.Random` que usan el greedy aleatorio, la búsqueda local y la búsqueda tabú, por lo que cada ejecución es reproducible.

1.5. Descripción del código

El código está escrito en Python y se organiza en los siguientes módulos:

- `main.py`: lee `parametros.txt`, carga cada instancia y lanza los algoritmos indicados en `ALGORITMO`, una vez por semilla.
- `extracion_datos.py`: lee el fichero `.tsp` y calcula la matriz de distancias con NumPy.
- `Extraccion_parametros.py`: lee los parámetros y genera las semillas por rotación.
- `Funcion_evaluacion.py`: función objetivo, evaluación factorizada del intercambio y aplicación del movimiento.
- `greedy.py`, `greedy_aleatorio.py`, `Busqueda_local.py` y `Busqueda_Tabu.py`: un módulo por algoritmo.
- `logs.py`: escritura de los ficheros de registro. Los movimientos se acumulan en memoria y se escriben por bloques, y el tiempo que se dedica a escribir se descuenta del tiempo de ejecución que se informa.

2. Algoritmos

2.1. Greedy

Para cada ciudad se calcula la suma de sus distancias al resto, se ordena el vector de pares ⟨ciudad, suma⟩ de menor a mayor y el recorrido es ese orden. Es determinista y cuesta $O(n^2 + n \log n)$.

Algoritmo 1 Greedy

Entrada: Matriz de distancias D de tamaño n
Salida: Recorrido π

1: $V \leftarrow [\,]$
2: **para** $i \leftarrow 0$ **hasta** $n-1$ **hacer**
3: añadir $\langle i, \sum_j D[i][j] \rangle$ a V
4: **fin para**
5: ordenar V por la suma de forma ascendente
6: $\pi \leftarrow$ ciudades de V en ese orden
7: **devolver** π

2.2. Greedy aleatorio

Parte del mismo vector ordenado, pero en cada paso elige al azar una de las $K = 5$ primeras ciudades que quedan, la añade al recorrido y la elimina del vector. Cuando quedan menos de K ciudades se elige entre las restantes.

Algoritmo 2 Greedy aleatorio

Entrada: D, generador aleatorio, K
Salida: Recorrido π

1: $V \leftarrow$ vector $\langle$ciudad, suma de distancias$\rangle$ ordenado ascendentemente
2: $\pi \leftarrow [\,]$
3: **mientras** V no está vacío **hacer**
4: $r \leftarrow$ entero aleatorio en $[0, \text{mín}(K, |V|) - 1]$
5: añadir la ciudad $V[r]$ a π y eliminarla de V
6: **fin mientras**
7: **devolver** π

2.3. Búsqueda local del primer mejor con DLB

Parte de una solución del greedy aleatorio. Recorre las posiciones con el bit DLB a 0 y, para cada una, prueba los intercambios con el resto de posiciones (de forma circular a partir de $i + 1$). En cuanto un intercambio mejora el coste lo aplica (primer mejor), reactiva los bits de las dos posiciones y vuelve a explorar la misma posición. Si ninguno mejora, desactiva su bit. Termina cuando todos los bits están a 1 (óptimo local respecto al operador de intercambio) o cuando se alcanzan 10.000 movimientos.

Algoritmo 3 Búsqueda local del primer mejor con DLB

Entrada: D, máximo de iteraciones M, K
Salida: Mejor recorrido encontrado

$\pi \leftarrow \text{GreedyAleatorio}(D, K)$; $c \leftarrow F(\pi)$
$dlb \leftarrow [0, \ldots, 0]$; $it \leftarrow 0$; $i \leftarrow 0$
mientras $it < M$ **y** algún $dlb[i] = 0$ **hacer**
  si $dlb[i] = 0$ **entonces**
    $mejora \leftarrow$ falso
    para $k \leftarrow 1$ **hasta** $n - 1$ **hacer**
      $j \leftarrow (i + k) \bmod n$
      si $\text{Factorizacion}(\pi, i, j, c) < c$ **entonces**
        intercambiar π_i y π_j; $c \leftarrow F(\pi)$; $it \leftarrow it + 1$
        $dlb[i] \leftarrow 0$; $dlb[j] \leftarrow 0$; $mejora \leftarrow$ verdadero
        salir del bucle ▷ *se vuelve a explorar la misma i*
      fin si
    fin para
    si no $mejora$ **entonces**
      $dlb[i] \leftarrow 1$; $i \leftarrow (i + 1) \bmod n$
    fin si
  si no
    $i \leftarrow (i + 1) \bmod n$
  fin si
fin mientras
devolver π

2.4. Búsqueda tabú

La búsqueda tabú amplía la búsqueda local para que no se detenga en el primer óptimo local: se ejecutan siempre las 10.000 iteraciones y, cuando ningún vecino permitido mejora la solución actual, se aplica el **mejor vecino no tabú** aunque empeore el coste ("la mejor de las peores"). Durante la búsqueda se mantienen tres soluciones: la *solución actual*, el *mejor del momento actual* (la mejor desde el último reinicio) y el *mejor global*.

Memorias. Se usa una única matriz T de enteros de tamaño $n \times n$:

- **Memoria a corto plazo (MCP)**, en el triángulo superior ($T[a][b]$ con $a < b$). Al intercambiar las posiciones i y j, que contienen las ciudades π_i y π_j, se guardan dos valores con la tenencia tabú $t = 10$: el par de posiciones $T[\text{mín}(i,j)][\text{máx}(i,j)] \leftarrow t$ y el par de ciudades $T[\text{mín}(\pi_i, \pi_j)][\text{máx}(\pi_i, \pi_j)] \leftarrow t$. Un movimiento es tabú, y se descarta al generar el entorno, si cualquiera de sus dos celdas es mayor que 0. No se usa criterio de aspiración. Para no recorrer la matriz entera en cada iteración, las celdas activas se guardan en un conjunto: en cada iteración se decrementan solo esas celdas y se eliminan las que llegan a 0.

- **Memoria a largo plazo (MLP)**, en el triángulo inferior ($T[a][b]$ con $a > b$). En cada iteración se incrementa en 1 la celda de cada arco (π_k, π_{k+1}) de la solución actual, es decir, n celdas por iteración. Como el problema es simétrico, el arco (u, v) y el (v, u) comparten celda.

Exploración del entorno. Es la misma que la de la búsqueda local (DLB y primer mejor), pero saltando los movimientos tabú. Mientras se exploran los vecinos se guarda el mejor vecino no tabú visto. Si se encuentra una mejora se aplica. Si todos los bits del DLB quedan a 1 sin mejora, se aplica ese mejor vecino. Tras aplicar cualquier movimiento solo se reactivan los bits de las dos posiciones intercambiadas, por lo que la siguiente exploración evalúa unos $2n$ vecinos en lugar de los $n^2/2$ del entorno completo.

Estancamiento y oscilación estratégica. Cada movimiento que no mejora al mejor del momento actual cuenta como movimiento de empeoramiento. Si se acumulan $0{,}05 \cdot 10.000 = 500$ consecutivos, se considera que la búsqueda se ha estancado y se genera una solución nueva: con probabilidad 0,5 por **intensificación** y con probabilidad 0,5 por **diversificación**. Ambos operadores construyen un recorrido desde una ciudad aleatoria y, en cada paso, eligen al azar entre las $K = 5$ ciudades no visitadas cuyo arco con la ciudad actual tiene **mayor** frecuencia en la MLP (intensificación: reconstruye los arcos que más han aparecido en las buenas soluciones) o **menor** frecuencia (diversificación: lleva la búsqueda a zonas poco visitadas). Después se reinician la MCP, el DLB, el contador de estancamiento y el mejor del momento actual; la MLP se conserva.

Algoritmo 4 Búsqueda tabú

Entrada: D, $M = 10.000$, $K = 5$, tenencia $t = 10$, estancamiento $e = 0{,}05$, oscilación $p = 0{,}5$

Salida: Mejor recorrido global

$\pi \leftarrow$ GreedyAleatorio(D, K); $\pi_{mom} \leftarrow \pi$; $\pi_{glob} \leftarrow \pi$
$T \leftarrow$ matriz $n \times n$ de ceros; $dlb \leftarrow [0, \ldots, 0]$; $it \leftarrow 0$; $est \leftarrow 0$
mientras $it < M$ **hacer**
    $(i, j) \leftarrow$ primer movimiento no tabú que mejora π usando el DLB
    si no existe **entonces**
        $(i, j) \leftarrow$ mejor movimiento no tabú evaluado ▷ *la mejor de las peores*
    fin si
    decrementar en 1 las celdas activas de la MCP
    T[posiciones i, j] $\leftarrow t$; T[ciudades π_i, π_j] $\leftarrow t$
    intercambiar π_i y π_j; $dlb[i] \leftarrow 0$; $dlb[j] \leftarrow 0$; $it \leftarrow it + 1$
    incrementar en la MLP los n arcos de π
    si $F(\pi) < F(\pi_{mom})$ **entonces**
        $\pi_{mom} \leftarrow \pi$; $est \leftarrow 0$
        si $F(\pi) < F(\pi_{glob})$ **entonces** $\pi_{glob} \leftarrow \pi$
        fin si
    si no
        $est \leftarrow est + 1$
    fin si
    si $est \geq e \cdot M$ **entonces**
        si aleatorio $< p$ **entonces**
            $\pi \leftarrow$ Intensificacion(T, K)
        si no
            $\pi \leftarrow$ Diversificacion(T, K)
        fin si
        $\pi_{mom} \leftarrow \pi$; reiniciar MCP, *dlb* y *est* ▷ *la MLP no se reinicia*
    fin si
fin mientras
devolver π_{glob}

Algoritmo 5 Construcción de una solución a partir de la MLP (intensificación y diversificación)

Entrada: MLP, K, modo $\in$ {intensificación, diversificación}
Salida: Recorrido π

```
actual ← ciudad aleatoria;   π ← [actual]
mientras |π| < n hacer
    f ← frecuencias en la MLP de los arcos (actual, v) para cada v no visitada
    si modo = intensificación entonces
        C ← las K ciudades no visitadas con mayor f
    si no
        C ← las K ciudades no visitadas con menor f
    fin si
    actual ← ciudad aleatoria de C;   añadir actual a π
fin mientras
devolver π
```

3. Experimentos y análisis de resultados

3.1. Parámetros y semillas

Todos los parámetros se leen de `parametros.txt`:

Parámetro	Valor	Algoritmos
Semilla inicial	1658743 (rotación de dígitos)	GA, BL, Tabú
Ejecuciones por instancia	3 (1 en el greedy, que es determinista)	Todos
K	5	GA, BL, Tabú
Máximo de iteraciones	10.000	BL, Tabú
Tenencia tabú	10	Tabú
Estancamiento	5 % de las iteraciones (500)	Tabú
Oscilación estratégica	50 %	Tabú

Tabla 1: Parámetros de las ejecuciones.

Como `main.py` encadena la rotación de la semilla de una ejecución a la siguiente y de un fichero al siguiente, cada algoritmo usa en cada instancia las semillas que se indican en el pie de su tabla de resultados. Los tiempos se midieron en el mismo equipo y no incluyen la escritura de los logs.

3.2. Resultados

Las Tablas 2 a 5 recogen, en el formato de la plantilla de la asignatura, el coste y el tiempo de cada ejecución junto con su media y desviación típica. La Tabla 6 resume la desviación media respecto al óptimo y el tiempo medio. Los óptimos utilizados son: ch130 = 6.110, pr144 = 58.537, a280 = 2.579, u1060 = 224.094 y d18512 = 645.238.

	ch130		pr144		a280		u1060		d18512	
	Coste	T (s)	Coste	T (s)	Coste	T (s)	Coste	T (s)	Coste	T (s)
Ejecución 1	40.397,47	0,000	640.770,87	0,000	30.296,82	0,001	5.670.064,52	0,002	51.161.687,44	0,160
Media	40.397,47	0,000	640.770,87	0,000	30.296,82	0,001	5.670.064,52	0,002	51.161.687,44	0,160
Desv. típica	–	–	–	–	–	–	–	–	–	–

Tabla 2: Resultados de Greedy (semillas por instancia, en orden de ejecución: ch130: 1658743; pr144: 1658743; a280: 1658743; u1060: 1658743; d18512: 1658743).

	ch130		pr144		a280		u1060		d18512	
	Coste	T (s)	Coste	T (s)	Coste	T (s)	Coste	T (s)	Coste	T (s)
Ejecución 1	40.164,55	0,001	651.173,54	0,001	29.454,85	0,003	5.485.445,65	0,024	51.931.042,22	8,570
Ejecución 2	44.011,38	0,001	688.277,45	0,001	30.282,71	0,002	5.432.360,65	0,024	51.597.813,86	9,044
Ejecución 3	40.963,54	0,001	645.174,67	0,001	30.237,78	0,002	5.726.073,12	0,024	50.934.175,26	8,556
Media	41.713,16	0,001	661.541,89	0,001	29.991,78	0,002	5.547.959,81	0,024	51.487.677,12	8,724
Desv. típica	2.030,02	0,000	23.347,15	0,000	465,54	0,000	156.517,63	0,000	507.477,59	0,278

Tabla 3: Resultados de Greedy aleatorio (semillas por instancia, en orden de ejecución: ch130: 7431658, 4316587, 3165874; pr144: 4316587, 3165874, 1658743; a280: 5874316, 8743165, 7431658; u1060: 1658743, 6587431, 5874316; d18512: 6587431, 5874316, 8743165).

	ch130		pr144		a280		u1060		d18512	
	Coste	T (s)	Coste	T (s)	Coste	T (s)	Coste	T (s)	Coste	T (s)
Ejecución 1	13.868,04	0,091	222.961,32	0,104	6.092,60	0,567	679.598,50	11,552	50.148.233,54	78,626
Ejecución 2	12.709,21	0,097	204.411,47	0,104	7.488,31	0,583	812.103,44	12,184	51.170.029,17	78,715
Ejecución 3	12.099,93	0,101	222.428,84	0,117	6.848,08	0,548	780.085,34	12,598	50.828.025,85	78,905
Media	12.892,39	0,096	216.600,54	0,108	6.809,66	0,566	757.262,43	12,111	50.715.429,52	78,749
Desv. típica	898,17	0,005	10.559,40	0,008	698,65	0,018	69.137,93	0,527	520.120,20	0,143

Tabla 4: Resultados de Búsqueda local (semillas por instancia, en orden de ejecución: ch130: 8743165, 7431658, 4316587; pr144: 7431658, 4316587, 3165874; a280: 6587431, 5874316, 8743165; u1060: 3165874, 1658743, 6587431; d18512: 1658743, 6587431, 5874316).

	ch130		pr144		a280		u1060		d18512	
	Coste	T (s)	Coste	T (s)	Coste	T (s)	Coste	T (s)	Coste	T (s)
Ejecución 1	8.603,99	4,407	113.961,24	4,516	4.404,22	9,935	815.904,79	14,523	50.822.608,96	77,483
Ejecución 2	9.141,59	4,785	121.553,71	5,174	4.730,26	8,817	852.455,12	13,933	50.163.359,62	77,310
Ejecución 3	9.010,83	4,332	117.268,02	4,676	4.191,06	8,810	730.441,48	16,158	50.565.042,72	76,319
Media	8.918,80	4,508	117.594,32	4,789	4.441,85	9,187	799.600,46	14,871	50.517.003,77	77,037
Desv. típica	280,37	0,243	3.806,74	0,343	271,56	0,648	62.619,53	1,153	332.239,72	0,628

Tabla 5: Resultados de Búsqueda tabú (semillas por instancia, en orden de ejecución: ch130: 4316587, 3165874, 1658743; pr144: 3165874, 1658743, 6587431; a280: 8743165, 7431658, 4316587; u1060: 6587431, 5874316, 8743165; d18512: 5874316, 8743165, 7431658).

	ch130		pr144		a280		u1060		d18512		Media	
	Desv (%)	T (s)	Desv (%)	T (s)	Desv (%)	T (s)	Desv (%)	T (s)	Desv (%)	T (s)	Desv (%)	T (s)
Greedy	561,17	0,000	994,64	0,000	1.074,75	0,001	2.430,22	0,002	7.829,12	0,160	2.577,98	0,033
Greedy aleatorio	582,70	0,001	1.030,13	0,001	1.062,92	0,002	2.375,73	0,024	7.879,64	8,724	2.586,22	1,750
Búsqueda local	111,00	0,096	270,02	0,108	164,04	0,566	237,92	12,111	7.759,96	78,749	1.708,59	18,326
Búsqueda tabú	45,97	4,508	100,89	4,789	72,23	9,187	256,81	14,871	7.729,20	77,037	1.641,02	22,079

Tabla 6: Desviación media respecto al óptimo y tiempo medio por algoritmo e instancia.

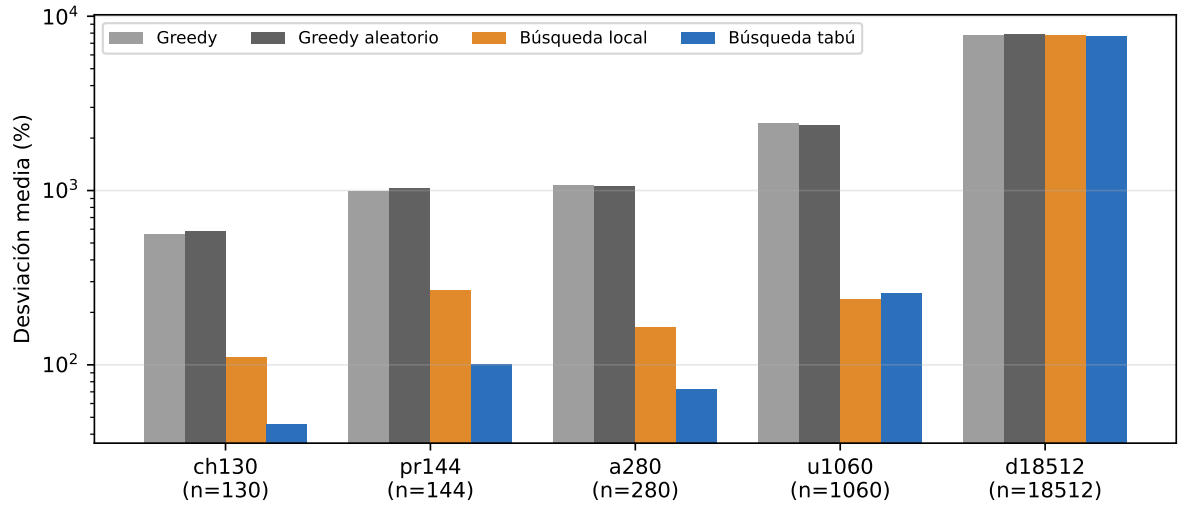


Figura 1: Desviación media respecto al óptimo por algoritmo e instancia (escala logarítmica).

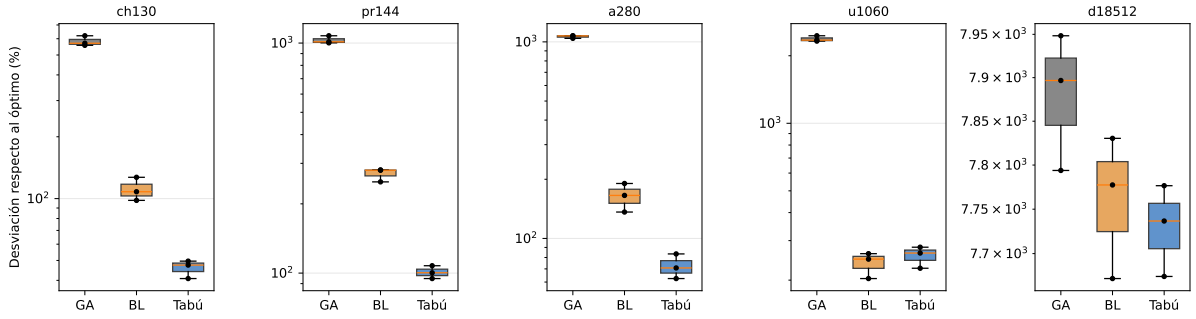


Figura 2: Distribución de la desviación de las ejecuciones del greedy aleatorio (GA), la búsqueda local (BL) y la búsqueda tabú. Cada punto es una ejecución.

3.3. Análisis de los algoritmos

Greedy y greedy aleatorio. Ambos dan recorridos entre 6 y 80 veces más largos que el óptimo (Tabla 6). La causa es el criterio de construcción: ordenar las ciudades por la suma de sus distancias al resto coloca primero las ciudades centrales y después las periféricas, pero no tiene en cuenta si dos ciudades consecutivas del recorrido están cerca entre sí. El resultado es un recorrido que salta de un extremo a otro del plano. La aleatoriedad del greedy aleatorio, al elegir entre las $K = 5$ siguientes, no corrige este defecto: su media queda muy cerca de la del greedy (entre un 2 % mejor y un 3 % peor según la instancia) y

solo añade dispersión (Figura 2). Su valor está en que es instantáneo y en que proporciona soluciones iniciales distintas para las búsquedas basadas en trayectorias.

Búsqueda local. Reduce la desviación del greedy aleatorio entre 3,8 y 10 veces en las cuatro instancias menores. La Tabla 7 muestra por qué es tan rápida en las pequeñas: en ch130, pr144 y a280 llega a un óptimo local tras unos 500–1.600 movimientos y se detiene en menos de un segundo, porque ningún intercambio mejora la solución. Ese óptimo local sigue lejos del óptimo global (98–270 % de desviación) porque el operador de intercambio es poco potente para el TSP: mover dos ciudades cambia cuatro arcos a la vez y casi nunca deshace un cruce entre aristas, que es el defecto típico de un mal recorrido (Figura 5).

Instancia	Ejec.	Movimientos	Motivo de parada
ch130	1	493	no se encontraron más movimientos de mejora
ch130	2	541	no se encontraron más movimientos de mejora
ch130	3	560	no se encontraron más movimientos de mejora
pr144	1	515	no se encontraron más movimientos de mejora
pr144	2	554	no se encontraron más movimientos de mejora
pr144	3	593	no se encontraron más movimientos de mejora
a280	1	1.634	no se encontraron más movimientos de mejora
a280	2	1.550	no se encontraron más movimientos de mejora
a280	3	1.509	no se encontraron más movimientos de mejora
u1060	1	9.383	no se encontraron más movimientos de mejora
u1060	2	9.340	no se encontraron más movimientos de mejora
u1060	3	9.866	no se encontraron más movimientos de mejora
d18512	1	10.000	alcanzado el máximo de movimientos
d18512	2	10.000	alcanzado el máximo de movimientos
d18512	3	10.000	alcanzado el máximo de movimientos

Tabla 7: Movimientos realizados por la búsqueda local y motivo de parada.

Búsqueda tabú. Es el mejor algoritmo en ch130, pr144 y a280, con una desviación media entre 2,3 y 2,7 veces menor que la de la búsqueda local. La Tabla 8 explica el motivo: en estas instancias alrededor de un tercio de las 10.000 iteraciones son movimientos de empeoramiento. Al aceptar "la mejor de las peores" y prohibir durante 10 iteraciones deshacer el movimiento, la búsqueda sale del óptimo local donde se habría detenido la búsqueda local y continúa descendiendo por otra zona. En 7 de las 9 ejecuciones la mejor solución global se encuentra después de la iteración 7.000, es decir, mucho después del punto en que la búsqueda local ya había terminado.

La Figura 3 muestra esta dinámica. En ch130 las dos ejecuciones representadas usan la misma semilla y parten de la misma solución, por lo que sus curvas coinciden mientras ambas descienden; la búsqueda local se detiene hacia la iteración 560 y el tabú sigue mejorando poco a poco. Los picos de la solución actual corresponden a los reinicios por estancamiento (Tabla 8): las soluciones por diversificación empiezan muy lejos del mejor coste, porque construyen el recorrido con los arcos menos usados, y la búsqueda vuelve a descender desde ellas. Los reinicios por intensificación parten de costes mucho más bajos, porque reutilizan los arcos que más se han repetido: en ch130 sus soluciones iniciales cuestan entre 21.270 y 24.400, frente a 51.470–55.690 las de diversificación.

Instancia	Ejec.	Empeoramientos	Reinicios (I/D)	Última mejora global	Coste final
ch130	1	3.702	3 (2/1)	7.598	8.603,99
ch130	2	3.387	4 (2/2)	7.942	9.141,59
ch130	3	3.556	4 (2/2)	9.047	9.010,83
pr144	1	3.236	5 (2/3)	4.259	113.961,24
pr144	2	3.588	4 (3/1)	9.522	121.553,71
pr144	3	3.376	4 (2/2)	9.006	117.268,02
a280	1	3.691	1 (1/0)	7.345	4.404,22
a280	2	2.845	3 (1/2)	4.572	4.730,26
a280	3	3.128	2 (1/1)	9.992	4.191,06
u1060	1	69	0 (0/0)	10.000	815.904,79
u1060	2	0	0 (0/0)	10.000	852.455,12
u1060	3	175	0 (0/0)	9.991	730.441,48
d18512	1	0	0 (0/0)	10.000	50.822.608,96
d18512	2	0	0 (0/0)	10.000	50.163.359,62
d18512	3	0	0 (0/0)	10.000	50.565.042,72

Tabla 8: Comportamiento interno de la búsqueda tabú (10.000 iteraciones por ejecución).

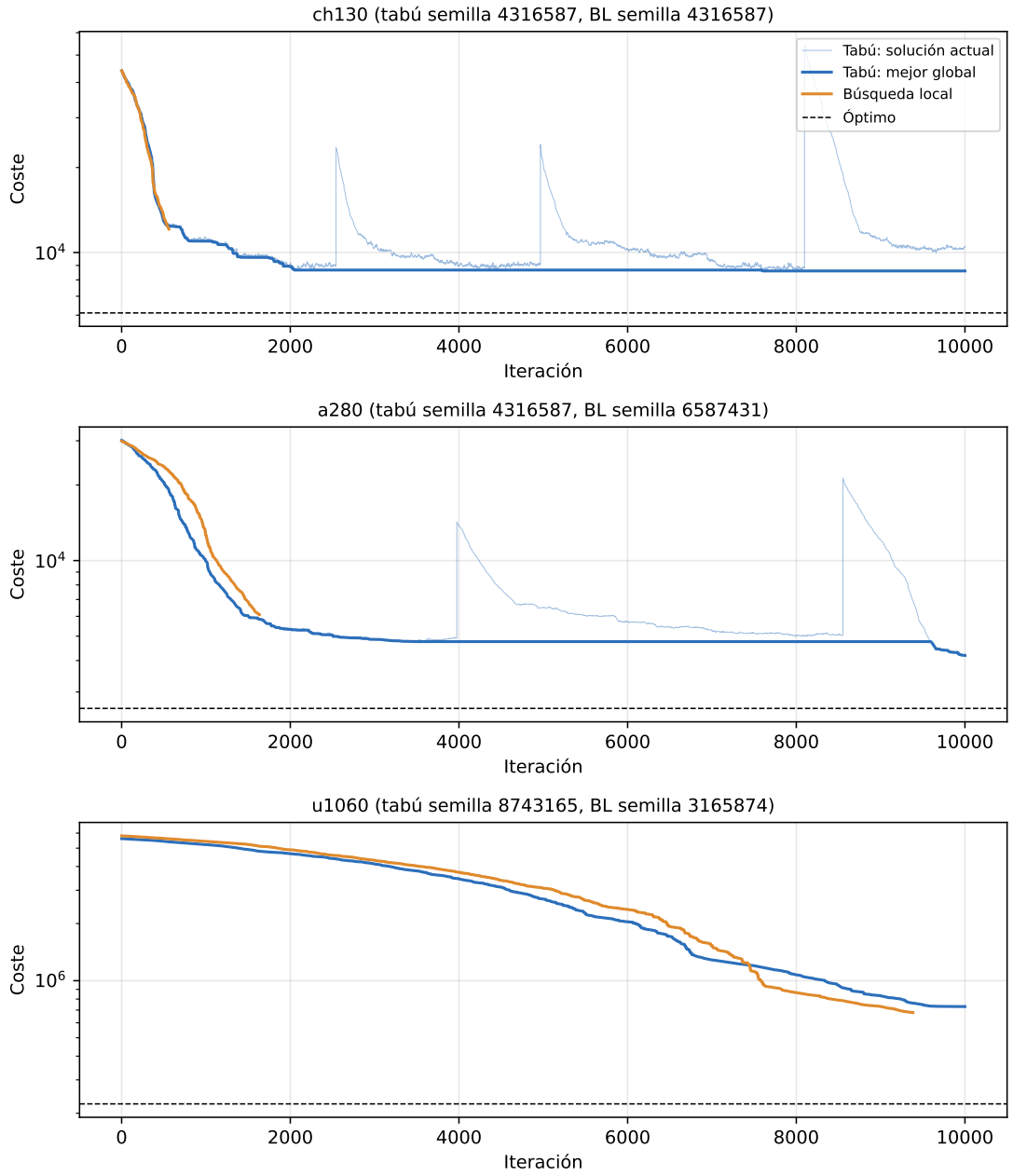


Figura 3: Convergencia de la mejor ejecución de la búsqueda tabú y de la búsqueda local en ch130, a280 y u1060 (coste en escala logarítmica). La línea fina es la solución actual del tabú y la discontinua, el óptimo.

En u1060 el comportamiento cambia. La búsqueda local necesita entre 9.340 y 9.870 movimientos para llegar a un óptimo local, casi todo el máximo de 10.000. Con el mismo presupuesto, el tabú dedica casi todas sus iteraciones a descender igual que la búsqueda local (entre 0 y 175 empeoramientos, ningún reinicio), así que sus mecanismos apenas llegan a actuar. En media queda por detrás (256,8 % frente a 237,9 %), pero esa diferencia la explican las semillas, que son distintas en cada algoritmo, y no el algoritmo. Para comprobarlo se ejecutaron ambos con las mismas tres semillas (1658743, 6587431 y 5874316): el tabú obtuvo un recorrido mejor en dos de ellas (763.691 frente a 812.103, y 852.455 frente a 881.699) y peor en la tercera (815.905 frente a 780.085). Además, como la MCP prohíbe durante 10 iteraciones algunos movimientos de mejora y no hay criterio

de aspiración, el descenso del tabú sigue un camino distinto al de la búsqueda local desde el principio.

En d18512 ocurre lo mismo de forma más acusada: 10.000 iteraciones son muy pocas para 18.512 ciudades. Todas las iteraciones del tabú son de mejora (ningún empeoramiento ni reinicio), y la búsqueda tabú (7.729,2 % de desviación media) y la búsqueda local (7.760,0 %) quedan prácticamente en el mismo punto, apenas por debajo del greedy aleatorio del que parten.

3.4. Tiempos, desviaciones e influencia del tamaño

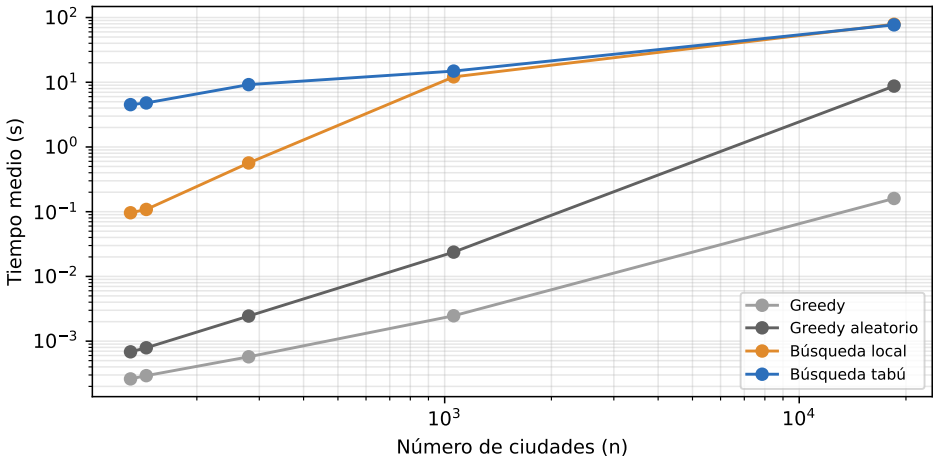


Figura 4: Tiempo medio por ejecución frente al número de ciudades (escalas logarítmicas).

El tamaño de la instancia influye de tres formas:

- **Coste de los greedy.** Calcular las sumas de distancias es $O(n^2)$, pero se hace con operaciones vectorizadas de NumPy, por lo que el greedy tarda 0,16 s incluso en d18512. El greedy aleatorio tarda 8,7 s en esa instancia porque reordena el vector de candidatos en cada inserción. La matriz de distancias de d18512 ocupa por sí sola alrededor de 1,3 GB.

- **Presupuesto de iteraciones.** El número de movimientos necesario para llegar a un óptimo local crece con n (unos 530 en ch130, 1.560 en a280 y 9.500 en u1060). Con un límite fijo de 10.000 iteraciones, en u1060 la búsqueda local apenas llega y en d18512 queda muy lejos: las tres ejecuciones de la búsqueda local y las tres del tabú se cortan en 10.000 movimientos con una desviación superior al 7.600 %, prácticamente la misma que la del greedy aleatorio del que parten. A partir de cierto tamaño lo que limita la calidad no es el algoritmo sino el número de iteraciones permitido.

- **Tiempo por iteración.** La evaluación factorizada es $O(1)$, pero cada iteración incluye operaciones $O(n)$ (recalcular el coste, actualizar la MLP) y una exploración del DLB que puede recorrer hasta n vecinos por posición. Por eso el tabú, que siempre agota las 10.000 iteraciones, tarda en las instancias pequeñas de 16 a 47 veces más que la búsqueda local, que se detiene antes. En u1060, donde ambas hacen casi todas las iteraciones, la diferencia baja a 1,2 veces (14,9 s frente a 12,1 s),

y en d18512, donde las dos agotan las 10.000, tardan prácticamente lo mismo (77,0 s frente a 78,7 s).

En resumen, la búsqueda tabú ofrece la mejor relación calidad/tiempo en las instancias de hasta unos cientos de ciudades: unos pocos segundos más que la búsqueda local a cambio de dividir su desviación entre 2,3 y 2,7. En las instancias grandes, con 10.000 iteraciones, ninguna búsqueda por trayectorias llega a desplegar sus mecanismos y las diferencias entre ellas dependen sobre todo de la semilla. Con solo tres ejecuciones por algoritmo no se ha aplicado ningún test estadístico, ya que no tendría potencia suficiente; las diferencias entre la búsqueda local y la tabú en las tres instancias pequeñas son, en cualquier caso, mucho mayores que la dispersión entre ejecuciones (Figura 2).

3.5. Análisis de la mejor solución

La mejor solución de todo el estudio, medida por su desviación, es la de la búsqueda tabú en ch130: un recorrido de longitud 8.603,99, un 40,8 % más largo que el óptimo (6.110). La Figura 5 la compara con la mejor solución de la búsqueda local en la misma instancia (12.099,93; 98,0 %).

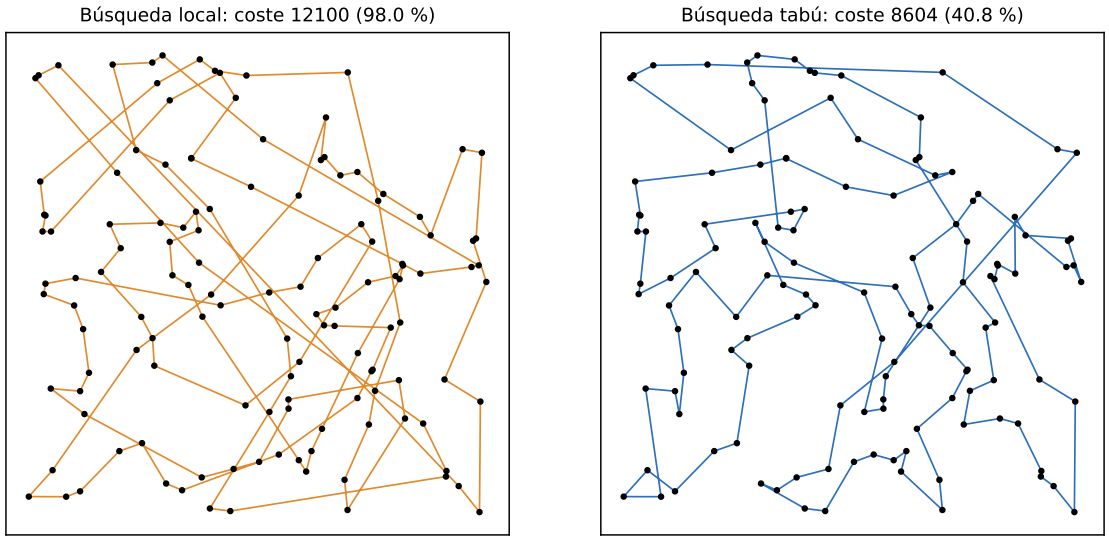


Figura 5: Mejor recorrido de la búsqueda local y de la búsqueda tabú en ch130.

El recorrido de la búsqueda local tiene numerosas aristas largas que cruzan el plano y se cortan entre sí. Es un óptimo local respecto al intercambio de dos ciudades: cada cruce solo se puede eliminar invirtiendo un tramo entero del recorrido, y eso no se consigue con un único intercambio que mejore el coste. El recorrido del tabú, en cambio, avanza casi siempre entre ciudades vecinas y recorre el plano por zonas, aunque conserva algunos cruces y saltos largos. Eso es lo que significa el 40,8 % restante: con el operador de intercambio, ni siquiera aceptando empeoramientos se llegan a eliminar todos los cruces en 10.000 iteraciones. Un operador 2-opt que invierta el tramo entre las dos posiciones eliminaría directamente esos cruces y reduciría esta distancia.

3.6. Ficheros de registro

Cada ejecución genera un fichero de log con los parámetros y la semilla, la solución inicial, cada movimiento aplicado con el coste anterior y el nuevo (de mejora en la búsqueda local; de mejora o de empeoramiento en el tabú, junto con el mejor del momento actual, el mejor global y el contador de estancamiento), cada reinicio por intensificación o diversificación con su nueva solución, y la solución y el coste finales. Los logs se pueden consultar en la carpeta indicada en el Anexo.

4. Referencias

- Material de la asignatura Metaheurísticas, Universidad de Jaén, curso 2026/2027.
- Reinelt, G. (1991). TSPLIB — A Traveling Salesman Problem Library. *ORSA Journal on Computing*, 3(4), 376–384. Óptimos de las instancias: `http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/`.
- Glover, F. y Laguna, M. (1997). *Tabu Search*. Kluwer Academic Publishers.

Anexo

A. Carpeta de Logs

Enlace de descarga a la carpeta con los Logs: `https://drive.google.com/drive/folders/1JOPyPUdfYNtIpM7BfSW0vkRCTi284ucn?usp=sharing`

La carpeta contiene un subdirectorio por algoritmo (`gre`, `grea`, `b_local` y `tabu`). Cada fichero se llama `<instancia>_<algoritmo>_<semilla>_ejecucion_<n>.log` y registra los parámetros, la solución inicial, cada movimiento con su coste (de mejora o, en el tabú, también de empeoramiento), los reinicios por estancamiento, la solución final y un resumen con el coste y el tiempo de la ejecución.